

Storage

Azure Storage ACCOUNTS



Blob Storage

- Object and Disk Storage
- Blob Storage Tiers
- Azure Search integration



File Storage

- SMB File Shares
- Attach to VMs as file shares
- Azure File Sync



Table Storage

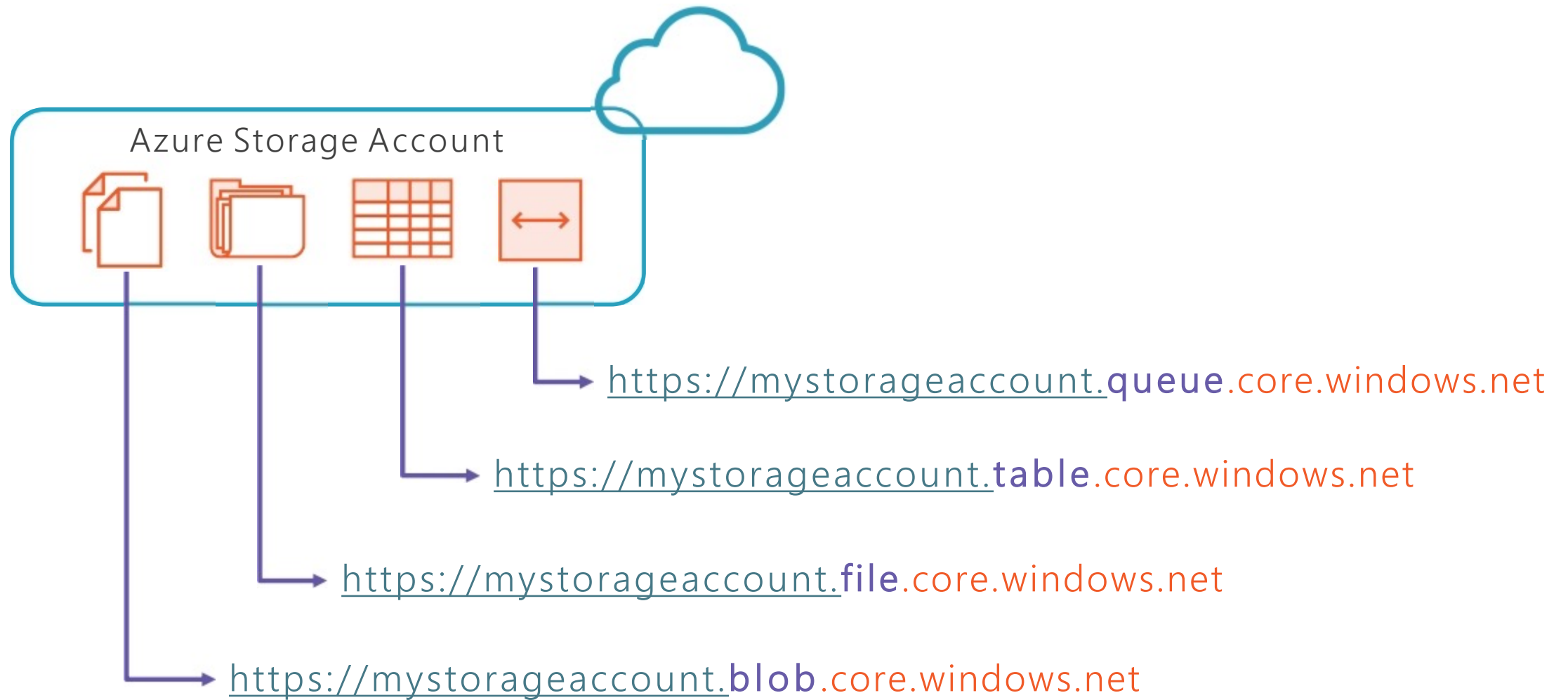
- NoSQL Data Store
- Schema-less design
- Azure Cosmos DB



Queue Storage

- Message based
- For building synchronous apps

Azure Storage account endpoints



Azure storage custom domain name

fstorage100 | Front Door and CDN ☆ ...
Storage account

domain × × <<

Security + networking

Front Door and CDN

Endpoints

Click on your endpoint below to manage and configure different features of your existing CDN endpoints. [Learn more about adding custom domains](#)

Host name	Profile name	Service type
-----------	--------------	--------------

New Endpoint

Service type

☒ Azure Front Door (Recommended)

☐ Azure CDN

Create new/use existing profile

☒ Create new

☐ Use existing

Profile name *

Endpoint name *

Endpoint host name

Origin host name *

fstorage100.blob.core.windows.net (Blob) ▾

Pricing tier *

[Learn more](#)

☒ Azure Front Door Standard
Content delivery optimized

☐ Azure Front Door Premium
Security optimized

Caching ⓘ

WAF ⓘ

☐ Enable caching

☐ Enable WAF

Create

Azure Storage accounts can be accessed using **custom domain names** setup with Azure Front Door.

Azure blob storage types



Block Blob

- 190.7 TB Max file size
- Composed of 4000MB blocks
- Insert, replace, delete blocks



Append Blob

- 195GB Max file size
- Can only append blocks
- Ideal for log/audit files



Page Blob

- 8TB max file size
- Frequent read/write operations
- Standard/Premium Storage

Use cases

Big Data & Analytics

- Data Lakes (ADLS Gen2)
- Machine Learning input/output
- All Big Data Pipelines

Cloud-Native Workloads

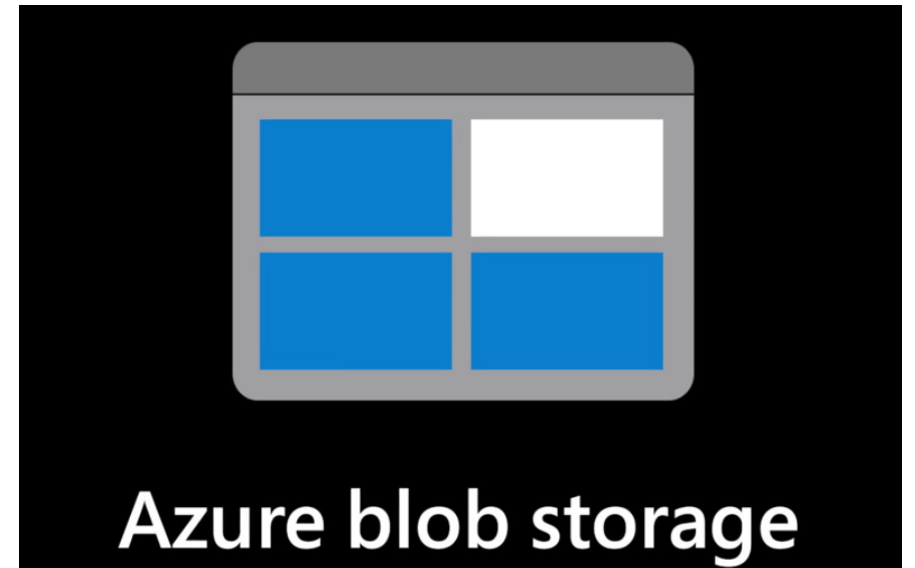
- Web apps, containers, and microservices
- Blob storage for static content, logs, and unstructured.

AI / ML & HPC

- Training datasets, inference results
- Temporary Compute Staging (e.g, scratch disks)

Long-Term & Archival Storage

- Cloud and archive tiers for compliance, backups, or rarely accessed data.



BLOB STORAGE tiers

	Hot	Cool	Archive
USE	Storing data that is frequently accessed or modified	Storing data that is infrequently accessed, but needs to remain accessible nonetheless	Storing data that is hardly ever accessed, but still needs to be kept
PROS	Low data access and transaction cost, and no minimum retention period	Storage costs are lower than the Hot tier	Has the lowest storage costs
CONS	The highest storage cost	Access and transaction costs are more than the Hot tier; also has a minimum data retention period of 30 days	The highest access and transaction cost and latency usually measured in hours; also has a minimum data retention period of 180 days

Demo – Create azure storage account with python

Setup and Prerequisites

Azure Subscription

Ensure you have an active subscription and required permissions.

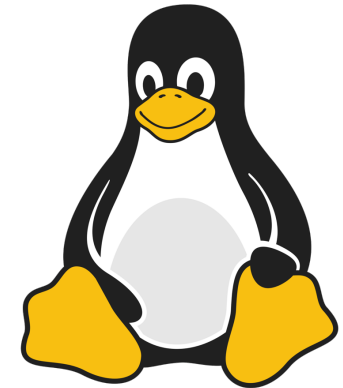
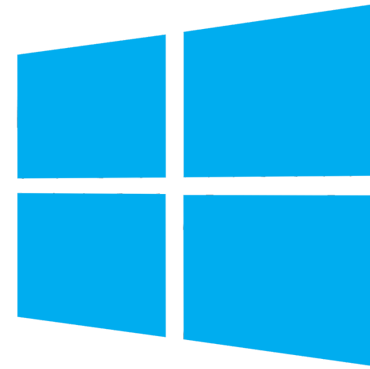
Python 3.8+ Environment

```
python --version
```

Install Required Packages

```
sudo apt-get update
```

```
pip install azure-identity azure-mgmt-resource azure-  
mgmt-storage azure-storage-blob
```



Demo – Create azure storage account with python

Create a project directory:

```
mkdir blobdemo & cd blobdemo  
touch provision_blob.py
```

Add the provisioning code

Run:

```
az login  
python provision_blob.py
```

```
import os  
from azure.identity import DefaultAzureCredential  
from azure.mgmt.resource import ResourceManagementClient  
from azure.mgmt.storage import StorageManagementClient  
  
SUBSCRIPTION_ID = "" # Replace with your Azure subscription ID  
LOCATION = "eastus"  
RG_NAME = "demo-rg"  
STORAGE_ACCOUNT_NAME = "demostorageacct2030" # Must be globally unique  
  
cred = DefaultAzureCredential()  
rg_client = ResourceManagementClient(cred, SUBSCRIPTION_ID)  
stg_client = StorageManagementClient(cred, SUBSCRIPTION_ID)  
  
rg_client.resource_groups.create_or_update(RG_NAME, {"location": LOCATION})  
print(f"Resource group '{RG_NAME}' created in {LOCATION}")  
  
stg_async = stg_client.storage_accounts.begin_create(  
    RG_NAME, STORAGE_ACCOUNT_NAME,  
    {  
        "location": LOCATION,  
        "sku": {"name": "Standard_LRS"},  
        "kind": "StorageV2",  
        "enable_https_traffic_only": True  
    }  
)  
stg_account = stg_async.result()  
print(f"Storage account '{STORAGE_ACCOUNT_NAME}' created")
```

Demo – Upload to azure storage account with python

Create an upload script

touch upload_blob.py

Add the file upload code

Run:

python upload_blob.py

```
# upload_blob.py

import os, uuid
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient

ACCOUNT_NAME = "demostorageacct2030" # Replace with your storage account name
ACCOUNT_URL = f"https://{ACCOUNT_NAME}.blob.core.windows.net"
CONTAINER_NAME = "demo-container"
LOCAL_FILE_PATH = "data/myfile.txt"
BLOB_NAME = f"{uuid.uuid4()}-{os.path.basename(LOCAL_FILE_PATH)}"

# Authenticate and connect
blob_service_client = BlobServiceClient(ACCOUNT_URL, credential=DefaultAzureCredential())

# 1. Create container if missing
try:
    container_client = blob_service_client.create_container(CONTAINER_NAME)
    print(f"Container '{CONTAINER_NAME}' created.")
except Exception:
    container_client = blob_service_client.get_container_client(CONTAINER_NAME)
    print(f"Using existing container '{CONTAINER_NAME}'.")

# 2. Upload blob
blob_client = container_client.get_blob_client(BLOB_NAME)
with open(LOCAL_FILE_PATH, "rb") as f:
    blob_client.upload_blob(f)
print(f"Uploaded blob: {BLOB_NAME}")
```

Pop quiz:

Scenario: Your team must ensure blobs can never be deleted or overwritten for 7 days, except under legal hold. Which feature supports this?

- A. Read-access geo-redundant storage (RA-GRS)
- B. Immutable Blob Storage policies with retention
- C. Soft-delete and versioning
- D. Premium GPv2 with snapshots enabled

Pop quiz:

Scenario: Your team must ensure blobs can never be deleted or overwritten for 7 days, except under legal hold.
Which feature supports this?

- A. Read-access geo-redundant storage (RA-GRS)
- B. Immutable Blob Storage policies with retention
- C. Soft-delete and versioning
- D. Premium GPv2 with snapshots enabled